

Tema 2.

Gramáticas Formales

Lenguajes Formales, Autómatas y Computabilidad

ETSI Informáticos, Universidad Politécnica de Madrid

María Poveda Villalón, Juan Castellanos, David Pérez del Rey



POLITÉCNICA

- Este material está basado en los apuntes de la asignatura “Lenguajes Formales, Autómatas y Computabilidad”. Algunas definiciones y ejemplos han sido tomados del libro “Teoría de lenguajes, gramáticas y autómatas” de Manuel Alfonseca, Justo Sancho y Miguel Martínez Orga.

- Definiciones previas
- Tipos de gramáticas
- Árbol de derivación
- Gramáticas bien formadas. Depuración de gramáticas
- Gramáticas de tipo 3. Gramáticas lineales

Definiciones

- **Gramática formal** es una cuádrupla $G = (\Sigma_T, \Sigma_N, S, P)$
 - Σ_T es el **alfabeto** de símbolos **terminales** (alfabeto principal)
 - Σ_N es el **alfabeto** de símbolos **NO terminales** (alfabeto auxiliar)
 - $\Sigma_T \cup \Sigma_N = \Sigma$ alfabeto de la gramática
 - $\Sigma_T \cap \Sigma_N = \emptyset$
 - **S** es el **axioma** de la gramática. Partimos de este símbolo para generar todo el lenguaje que genera una gramática.
 - $S \in \Sigma_N$
 - **P** conjunto de **producciones** (finito)
 - Producciones $u ::= v$ con $v \in \Sigma^*$
 - Con dos condiciones
 1. $u \in \Sigma^+$ (u no puede ser palabra vacía)
 2. $u = xAy$ donde $A \in \Sigma_N$, $x, y \in \Sigma^*$ (al menos un símbolo no terminal a la izquierda de la producción)

■ Derivación directa ($v \rightarrow w$)

- $(v, w) \in \Sigma^*$, con P conjunto de producciones
- Se dice que “w es derivación directa de v”
- Si existen 2 palabras “z” e “u” tales que
 - $v = z\mathbf{x}u$ y $w = z\mathbf{y}u$ siendo $\mathbf{x}::=\mathbf{y} \in P$

■ Derivación ($v \rightarrow_+ w$)

- $\exists$ secuencia de palabras $u_0, u_1 \dots u_n (n > 0)$
- Derivación de longitud n


$$\begin{array}{l} v = u_0 \\ u_0 \rightarrow u_1 \\ u_1 \rightarrow u_2 \\ \dots \\ u_{n-1} \rightarrow u_n \\ w = u_n \end{array}$$

■ Relación de Thue ($v \rightarrow_* w$)

- $v \rightarrow_* w$ si se verifica que $v \rightarrow_+ w$ o $v = w$

■ Toda palabra se deriva de sí misma con derivación de longitud 0

- $G = (\Sigma_T, \Sigma_N, S, P)$
- $\Sigma_T = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- $\Sigma_N = \{A, B\}$
- $S = A$
- $P = \{$
 $A::=AB; A::=B$
 $B::=0; B::=1; B::=2; B::=3; B::=4; B::=5; B::=6; B::=7; B::=8; B::=9\}$

- $G = (\Sigma_T, \Sigma_N, S, P)$

- $\Sigma_T = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

- $\Sigma_N = \{A, B\}$

- $S = A$

- $P = \{$

$A ::= AB; A ::= B$

$B ::= 0; B ::= 1; B ::= 2; B ::= 3; B ::= 4; B ::= 5; B ::= 6; B ::= 7; B ::= 8; B ::= 9\}$

Para generar $A \rightarrow 42$

$A \rightarrow AB \rightarrow A2 \rightarrow B2 \rightarrow 42$

- Si se tiene
 - $u ::= v$
 - $u ::= w$
- Entonces se puede notar de la siguiente forma
 - $u ::= v \mid w$
- Ejemplo de la gramática anterior:
- $P = \{$
 - $A ::= AB \mid B$
 - $B ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9\}$

■ Forma sentencial

- $x \in \Sigma^*$ es forma sentencial de $G \Leftrightarrow S \rightarrow *x$

■ Sentencia o palabra

- $x \in \Sigma^*$ es palabra o sentencia de $G \Leftrightarrow x$ es forma sentencial & $x \in \Sigma_T^*$

■ Lenguaje generado por la gramática G

- $L(G) = \{x / x \in \Sigma_T^* \wedge S \rightarrow *x\}$
- El conjunto de sentencias o palabras de G

- Dada la $G = (\Sigma_T = \{0, 1\}, \Sigma_N = \{S\}, S, P = \{S ::= \lambda \mid 00S11\})$

- Podemos expresar el lenguaje generado por G de las siguientes formas:

- $L(G) = \{0^{2n}1^{2n} \mid n \geq 1\} \cup \{\lambda\}$
- $L(G) = \{0^{2n}1^{2n} \mid n \geq 0\}$

- Ejemplos de derivaciones

- $S \rightarrow 00S11 \rightarrow 00 \lambda 11 \rightarrow 0011$
- $S \rightarrow 00S11 \rightarrow 0000S1111 \rightarrow 0000 \lambda 1111 \rightarrow 00001111$

- Dada la $G = (\Sigma_T = \{a, b, c\}, \Sigma_N = \{S, X\}, S, P)$

- $P = \{$

 - $S ::= aX$

 - $X ::= bbX \mid c\}$

- Podemos expresar el lenguaje generado por G de la siguiente forma:

 - $L(G) = \{ab^{2^n}c \mid n \geq 0\}$

- Ejemplos de derivaciones

 - $S \rightarrow aX \rightarrow abbX \rightarrow abbbbX \rightarrow abbbbc$

- Una **gramática** G se llama "**recursiva**" $\Leftrightarrow \exists A \in \Sigma_N, x, y \in \Sigma^*$ tal que $A \rightarrow_+ xAy$
 - Si $x=\lambda$ ($A \rightarrow_+ Ay$) se dice que G es "recursiva a izquierdas" en A
 - Si $y=\lambda$ ($A \rightarrow_+ xA$) se dice que G es "recursiva a derechas" en A
- Una regla de **producción** es **recursiva** si tiene la forma $A ::= xAy$
 - Si $x=\lambda$ ($A ::= Ay$) se dice que es "recursiva a izquierdas"
 - Si $y=\lambda$ ($A ::= xA$) se dice que es "recursiva a derechas"
- Si un **lenguaje** es **infinito**, la G que lo representa tiene que ser **recursiva**

- Dada la $G = (\Sigma_T = \{a, b\}, \Sigma_N = \{A, B\}, A, P)$

- $P = \{$

 - $A ::= aB \mid a$

 - $B ::= bA\}$

- Podemos expresar el lenguaje generado por G de las siguientes formas:

 - $L(G) = \{(ab)^n a \mid n \geq 0\}$

- Ejemplos de derivaciones

 - $A \rightarrow aB \rightarrow abA \rightarrow abaB \rightarrow ababA \rightarrow ababa$

Tipos de gramáticas

- Chomsky clasificó las gramáticas en cuatro grandes grupos (G_0, G_1, G_2, G_3) cada uno de los cuales incluye a los siguientes
- **Jerarquía de Chomsky:**
 - $G_3 \subset G_2 \subset G_1 \subset G_0$
 - $L_0 \supsetneq L_1 \supsetneq L_2 \supsetneq L_3$
- Se dice que dos **gramáticas** son **equivalentes** si describen el mismo lenguaje
- Dadas las gramáticas G y G'
 - $G \equiv G' \Leftrightarrow L(G) = L(G')$

Gramáticas tipo 0

- Reglas de producción del tipo (sin ninguna restricción adicional)
 - $u ::= v$ donde
 - $u \in \Sigma^+$ (u no puede ser λ)
 - $v \in \Sigma^*$
 - $u = xAy$
 - $x, y \in \Sigma^*$ (pueden ser λ)
 - $A \in \Sigma_N$ (A es no terminal)
- Los lenguajes representados por estas gramáticas se llaman “lenguajes sin restricciones”. L_0
- Pueden ser compresoras, en las derivaciones puede disminuir la longitud de las sucesivas palabras

- Se puede demostrar que todo lenguaje generado por gramática de Tipo 0 se puede generar por una **gramática de estructura de frases**.
- Reglas de producción de gramática de estructura de frases son del tipo:
 - $xAy ::= xvy$ donde
 - $x, y, v \in \Sigma^*$ (pueden ser λ)
 - $A \in \Sigma_N$ (A es no terminal)
- Teorema: Gramática G de Tipo 0 $\Rightarrow \exists G'$ en forma de estructura de frases tal que $L(G) = L(G')$
 - Es decir $G \equiv G'$

Ejemplo 1 Ejercicio gramática de estructura de frase

■ Dada la $G = (\Sigma_T = \{a, b\}, \Sigma_N = \{A, B, C\}, A, P)$

■ $P = \{$

$A ::= aABC \mid abC$

$CB ::= BC$ ← Esta regla no está en forma de estructura de frases

$bB ::= bb$

$bC ::= b\}$

$xAy ::= xvy$ donde

$x, y, v \in \Sigma^*$ (pueden ser λ)

$A \in \Sigma_N$ (A es no terminal)

Ejercicio estructura de frases

Indicar si las siguientes producciones sobre una gramática con $\Sigma_T = \{0, 1\}$, $\Sigma_N = \{S, A, B\}$ están en estructura de frases o no.

- a) $A ::= 0A01B1$
- b) $A ::= 0B$
- c) $AB ::= BA$
- d) $AB ::= SB$
- e) $AB ::= BS$
- f) $AB ::= SS$
- g) $01BA0 ::= 010$
- h) $01BA0 ::= 01A0$
- i) $01BA0 ::= 01B0$
- j) $01BA0 ::= BA$

Para casa

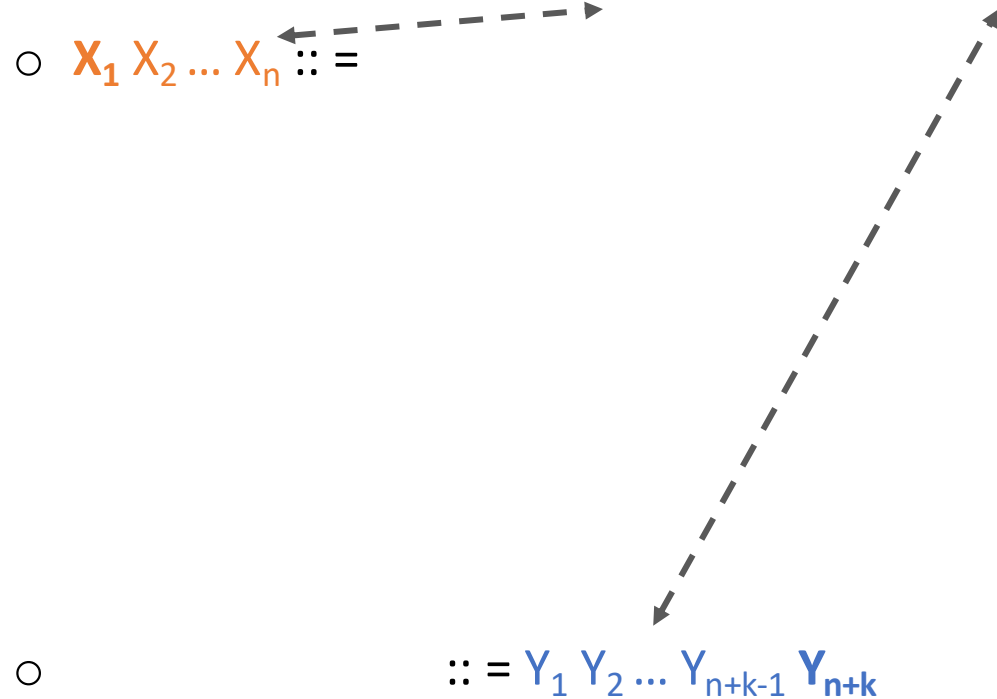
Gramáticas Tipo 0 – Estructura de frases

- Paso 1: Si en las producciones afectadas hay símbolos terminales b_i
 - Sustituir en **todas** las producciones b_i por X_i (símbolos no terminales **nuevos**)
 - Añadir $X_i ::= b_i$ al conjunto de producciones

- Paso 2: $u ::= v$ no sigue la estructura de frases

- Caso 1: Si $|u| \leq |v| \rightarrow X_1 X_2 \dots X_n ::= Y_1 Y_2 \dots Y_{n+k}$

○ $X_1 X_2 \dots X_n ::=$



○

$$::= Y_1 Y_2 \dots Y_{n+k-1} Y_{n+k}$$

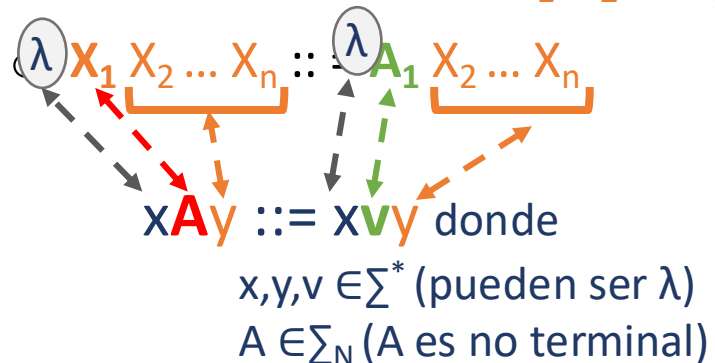
Gramáticas Tipo 0 – Estructura de frases

- Paso 1: Si en las producciones afectadas hay símbolos terminales b_i
 - Sustituir en **todas** las producciones b_i por X_i (símbolos no terminales **nuevos**)
 - Añadir $X_i ::= b_i$ al conjunto de producciones

- Paso 2: $u ::= v$ no sigue la estructura de frases

$A_1, A_2 \dots A_n \dots A_{n+k}$ son símbolos no terminales nuevos

- Caso 1: Si $|u| \leq |v| \rightarrow X_1 X_2 \dots X_n ::= Y_1 Y_2 \dots Y_{n+k}$



○

$::= Y_1 Y_2 \dots Y_{n+k-1} Y_{n+k}$

Gramáticas Tipo 0 – Estructura de frases

- Paso 1: Si en las producciones afectadas hay símbolos terminales b_i
 - Sustituir en **todas** las producciones b_i por X_i (símbolos no terminales **nuevos**)
 - Añadir $X_i ::= b_i$ al conjunto de producciones

- Paso 2: $u ::= v$ no sigue la estructura de frases

$A_1, A_2 \dots A_n \dots A_{n+k}$ son símbolos no terminales nuevos

- Caso 1: Si $|u| \leq |v| \rightarrow X_1 X_2 \dots X_n ::= Y_1 Y_2 \dots Y_{n+k}$

- $X_1 X_2 \dots X_n ::= A_1 X_2 \dots X_n$

- $A_1 X_2 \dots X_n ::= A_1 A_2 \dots X_n$

-

- $X A y ::= X v y$ donde

$x, y, v \in \Sigma^*$ (pueden ser λ)

- $A \in \Sigma_N$ (A es no terminal)

-

-

-

$$::= Y_1 Y_2 \dots Y_{n+k-1} Y_{n+k}$$

Gramáticas Tipo 0 – Estructura de frases

- Paso 1: Si en las producciones afectadas hay símbolos terminales b_i
 - Sustituir en **todas** las producciones b_i por X_i (símbolos no terminales **nuevos**)
 - Añadir $X_i ::= b_i$ al conjunto de producciones

- Paso 2: $u ::= v$ no sigue la estructura de frases

- Caso 1: Si $|u| \leq |v| \rightarrow X_1 X_2 \dots X_n ::= Y_1 Y_2 \dots Y_{n+k}$

○ $X_1 X_2 \dots X_n ::= A_1 X_2 \dots X_n$

○ $A_1 X_2 \dots X_n ::= A_1 A_2 \dots X_n$

○ ...

○ $A_1 A_2 \dots A_{n-1} X_n \overset{\lambda}{\circlearrowleft} ::= A_1 A_2 \dots A_{n-1} A_n \dots A_{n+k} \overset{\lambda}{\circlearrowleft}$

○

○

○

○

$x A y ::= x v y$ donde
 $x, y, v \in \Sigma^*$ (pueden ser λ)
 $A \in \Sigma_N$ (A es no terminal)
 $::= Y_1 Y_2 \dots Y_{n+k-1} Y_{n+k}$

$A_1, A_2 \dots A_n \dots A_{n+k}$ son símbolos no terminales nuevos

Cambio símbolos 1 a 1 hasta longitud n (la de partida) con símbolos no terminales **nuevos**

En la última transformación añadido todos los símbolos nuevos necesarios hasta $n+k$

Gramáticas Tipo 0 – Estructura de frases

- Paso 1: Si en las producciones afectadas hay símbolos terminales b_i
 - Sustituir en **todas** las producciones b_i por X_i (símbolos no terminales **nuevos**)
 - Añadir $X_i ::= b_i$ al conjunto de producciones

- Paso 2: $u ::= v$ no sigue la estructura de frases

- Caso 1: Si $|u| \leq |v| \rightarrow X_1 X_2 \dots X_n ::= Y_1 Y_2 \dots Y_{n+k}$

- $X_1 X_2 \dots X_n ::= A_1 X_2 \dots X_n$
- $A_1 X_2 \dots X_n ::= A_1 A_2 \dots X_n$
- ...
- $A_1 A_2 \dots A_{n-1} X_n ::= A_1 A_2 \dots A_{n-1} A_n \dots A_{n+k}$
- $A_1 A_2 \dots A_n \dots A_{n+k} ::= Y_1 A_2 \dots A_{n+k}$
- $Y_1 A_2 \dots A_n \dots A_{n+k} ::= Y_1 Y_2 \dots A_{n+k}$
- ...
- $Y_1 Y_2 \dots Y_{n+k-1} A_{n+k} ::= Y_1 Y_2 \dots Y_{n+k-1} Y_{n+k}$

$A_1, A_2 \dots A_n \dots A_{n+k}$ son símbolos no terminales nuevos

Cambio símbolos 1 a 1 hasta longitud n (la de partida) con símbolos no terminales **nuevos**

En la última transformación añadido todos los símbolos nuevos necesarios hasta $n+k$

Cambio símbolos 1 a 1 hasta conseguir la palabra **objetivo**

Ejemplo 1 Ejercicio gramática de estructura de frase

- Dada la $G = (\Sigma_T = \{a, b\}, \Sigma_N = \{A, B, C\}, A, P)$

- $P = \{$

$A ::= aABC \mid abC$

$CB ::= BC$ ← Esta regla no está en forma de estructura de frases

$bB ::= bb$

$bC ::= b\}$

$xAy ::= xvy$ donde

$x, y, v \in \Sigma^*$ (pueden ser λ)

$A \in \Sigma_N$ (A es no terminal)

- Se sustituye la regla $CB ::= BC$ por una serie de reglas nuevas:

$_CB ::= _XB$ ← X, Y son símbolos no terminales nuevos

$XB_ ::= XY_$

$_XY ::= _BY$

$BY_ ::= BC_$

- De manera que se tiene la nueva gramática:

Ejemplo 1 Ejercicio gramática de estructura de frase

- $G' = (\Sigma_T = \{a, b\}, \Sigma_N = \{A, B, C, X, Y\}, A, P')$

- $P' = \{$

$A ::= aABC \mid abC$

$CB ::= XB$

$XB ::= XY$

$XY ::= BY$

$BY ::= BC$

$bB ::= bb$

$bC ::= b\}$

- El lenguaje representado es $\{a^n b^n, n \geq 1\}$

Gramáticas Tipo 0 – Estructura de frases

■ Paso 2: $u ::= v$ no sigue la estructura de frases

- Caso 2: Si $|u| > |v| \rightarrow X_1 X_2 \dots X_n X_{n+1} \dots X_{n+k} ::= Y_1 Y_2 \dots Y_n$

- $X_1 X_2 \dots X_n X_{n+1} \dots X_{n+k} ::=$

-

-

-

-

-

-

-

-

-

-

-

$::= Y_1 Y_2 \dots Y_n$

Gramáticas Tipo 0 – Estructura de frases

■ Paso 2: $u ::= v$ no sigue la estructura de frases

$A_1, A_2 \dots A_n \dots A_{n+k}$ son símbolos
no terminales nuevos

• Caso 2: Si $|u| > |v| \rightarrow X_1 X_2 \dots X_n X_{n+1} \dots X_{n+k} ::= Y_1 Y_2 \dots Y_n$

○ $X_1 X_2 \dots X_n X_{n+1} \dots X_{n+k} ::= A_1 X_2 \dots X_n X_{n+1} \dots X_{n+k}$

○ $A_1 X_2 \dots X_n X_{n+1} \dots X_{n+k} ::= A_1 A_2 \dots X_n X_{n+1} \dots X_{n+k}$

○ ...

○ $A_1 A_2 \dots A_{n-1} X_n X_{n+1} \dots X_{n+k} ::= A_1 A_2 \dots A_n X_{n+1} \dots X_{n+k}$

○ $A_1 A_2 \dots A_n X_{n+1} \dots X_{n+k-1} X_{n+k} \lambda = A_1 A_2 \dots A_n X_{n+1} \dots X_{n+k-1} \lambda \lambda$

Cambio símbolos 1 a 1 hasta
longitud n (la que “se busca”)

$x A y ::= x v y$ donde

$x, y, v \in \Sigma^*$ (pueden ser λ)

$A \in \Sigma_N$ (A es no terminal)

$::= Y_1 Y_2 \dots Y_n$

Gramáticas Tipo 0 – Estructura de frases

■ Paso 2: $u ::= v$ no sigue la estructura de frases

$A_1, A_2 \dots A_n \dots A_{n+k}$ son símbolos
no terminales nuevos

• Caso 2: Si $|u| > |v| \rightarrow X_1 X_2 \dots X_n X_{n+1} \dots X_{n+k} ::= Y_1 Y_2 \dots Y_n$

○ $X_1 X_2 \dots X_n X_{n+1} \dots X_{n+k} ::= A_1 X_2 \dots X_n X_{n+1} \dots X_{n+k}$

○ $A_1 X_2 \dots X_n X_{n+1} \dots X_{n+k} ::= A_1 A_2 \dots X_n X_{n+1} \dots X_{n+k}$

○ ...

○ $A_1 A_2 \dots A_{n-1} X_n X_{n+1} \dots X_{n+k} ::= A_1 A_2 \dots A_n X_{n+1} \dots X_{n+k}$

○ $A_1 A_2 \dots A_n X_{n+1} \dots X_{n+k-1} X_{n+k} ::= A_1 A_2 \dots A_n X_{n+1} \dots X_{n+k-1}$

○ $A_1 A_2 \dots A_n X_{n+1} \dots X_{n+k-2} X_{n+k-1} ::= A_1 A_2 \dots A_n X_{n+1} \dots X_{n+k-2}$

○ ...

○ $A_1 A_2 \dots A_n X_{n+1} ::= A_1 A_2 \dots A_n$

○ $A_1 A_2 \dots A_n ::= Y_1 A_2 \dots A_n$

○ $Y_1 A_2 \dots A_n ::= Y_1 Y_2 \dots A_n$

○ ...

○ $Y_1 Y_2 \dots Y_{n-1} A_n ::= Y_1 Y_2 \dots Y_n$

Cambio símbolos 1 a 1 hasta
longitud n (la que “se busca”)

Elimino símbolos 1 a 1 por la
derecha hasta llegar a palabra
de longitud n (la que “se busca”)

Cambio símbolos 1 a 1 hasta
conseguir la palabra objetivo

Ejemplo 2 Ejercicio gramática de estructura de frase

■ Dada la $G = (\Sigma_T = \{a, b, c\}, \Sigma_N = \{S, A\}, S, P)$

■ $P = \{$

$S ::= aSb$

$Sb ::= bA$

$abA ::= c\}$

$xAy ::= xvy$ donde

$x, y, v \in \Sigma^*$ (pueden ser λ)

$A \in \Sigma_N$ (A es no terminal)

Estas reglas no están en forma de estructura de frases

■ Se introducen símbolos no terminales para los símbolos terminales

$U ::= a$

$V ::= b$

$W ::= c$

U, V y W son símbolos no terminales nuevos

■ Se añaden las nuevas reglas de producción a P sustituyendo los terminales por los nuevos no terminales.

Ejemplo 2 Ejercicio gramática de estructura de frase

■ $P = \{$

$S ::= USV$

$SV ::= VA$

$UVA ::= W$

$U ::= a$

$V ::= b$

$W ::= c\}$

Estas reglas no están en forma de estructura de frases

■ Se sustituye la regla $SV ::= VA$ por una serie de reglas nuevas:

$SV ::= XV$

$XV ::= XY$

$XY ::= VY$

$VY ::= VA$

Ejemplo 2 Ejercicio gramática de estructura de frase

- Se sustituye la regla $UVA ::= W$ por una serie de reglas nuevas:

- $k=2$ y $n=1$

$UVA ::= LVA$

$LVA ::= LV$

$LV ::= L$

$L ::= W$

- De manera que se tiene la nueva gramática:

■ Paso 2: $u ::= v$ no sigue la estructura de frases

- Caso 2: Si $|u| > |v| \rightarrow X_1 X_2 \dots X_n X_{n+1} \dots X_{n+k} ::= Y_1 Y_2 \dots Y_n$
 - $X_1 X_2 \dots X_n X_{n+1} \dots X_{n+k} ::= A_1 X_2 \dots X_n X_{n+1} \dots X_{n+k}$
 - $A_1 X_2 \dots X_n X_{n+1} \dots X_{n+k} ::= A_1 A_2 \dots X_n X_{n+1} \dots X_{n+k}$
 - ...
 - $A_1 A_2 \dots A_{n-1} X_n X_{n+1} \dots X_{n+k} ::= A_1 A_2 \dots A_n X_{n+1} \dots X_{n+k}$
 - $A_1 A_2 \dots A_n X_{n+1} \dots X_{n+k} ::= A_1 A_2 \dots A_n X_{n+1} \dots X_{n+k-1}$
 - $A_1 A_2 \dots A_n X_{n+1} \dots X_{n+k-1} ::= A_1 A_2 \dots A_n X_{n+1} \dots X_{n+k-2}$
 - ...
 - $A_1 A_2 \dots A_n X_{n+1} ::= A_1 A_2 \dots A_n$
 - $A_1 A_2 \dots A_n ::= Y_1 A_2 \dots A_n$
 - $Y_1 A_2 \dots A_n ::= Y_1 Y_2 \dots A_n$
 - ...
 - $Y_1 Y_2 \dots Y_{n-1} A_n ::= Y_1 Y_2 \dots Y_n$

Ejemplo 2 Ejercicio gramática de estructura de frase

- $G' = (\Sigma_T = \{a, b, c\}, \Sigma_N = \{S, A, X, Y, U, V, W, L\}, S, P)$

- $P = \{$

$S ::= USV$

$SV ::= XV$

$XV ::= XY$

$XY ::= VY$

$VY ::= VA$

$UVA ::= LVA$

$LVA ::= LV$

$LV ::= L$

$L ::= W$

$U ::= a$

$V ::= b$

$W ::= c\}$

- El lenguaje representado es $\{a^n cb^n, n \geq 0\}$

- Dada la $G = (\Sigma_T = \{a, b, c\}, \Sigma_N = \{S, A\}, S, P)$
- $P = \{$
 - $S ::= baA$
 - $abA ::= cb$
 - $aS ::= Aabb\}$
- Poner en forma de estructura de frases

Gramáticas tipo 1

- Gramáticas sensibles al contexto
- Reglas de producción del tipo
 - $xAy ::= xvy$ donde
 - $v \in \Sigma^+$
 - $x, y \in \Sigma^*$ (pueden ser λ)
 - $A \in \Sigma_N$ (A es no terminal)
- Los lenguajes representados por estas gramáticas se llaman “lenguajes dependientes del contexto”. L_1
- v no puede ser $\lambda \rightarrow$ No compresoras
 - Excepción si $S ::= \lambda$
- En estas gramática no disminuye la longitud de las palabras:
 - $\forall u, v \in \Sigma^* / u \rightarrow v \Rightarrow |u| \leq |v|$

- $G = (\Sigma_T = \{a, b\}, \Sigma_N = \{S\}, S, P)$

- $P = \{S ::= aSb \mid ab\}$

- $L(G) = \{a^n b^n \mid n \geq 1\}$

Gramáticas tipo 2

- Gramáticas de contexto libre
- Reglas de producción del tipo
 - $A ::= v$ donde
 - $v \in \Sigma^+$ (v no puede ser λ)
 - $A \in \Sigma_N$ (A es no terminal)
 - Puede contener: $S ::= \lambda$
- Los lenguajes representados por estas gramáticas se llaman “lenguajes independientes del contexto”. L_2
- Gramática no compresora
 - Si S figura en la derecha de alguna regla de producción y $S ::= \lambda$, dejará de ser no compresora

Ejemplo de gramática tipo 2

- $G = (\Sigma_T = \{a, b\}, \Sigma_N = \{S\}, S, P)$
 - $P = \{S ::= aSb \mid ab\}$

- $L(G) = \{a^n b^n \mid n \geq 1\}$

- Es la misma gramática que el ejemplo de Tipo 1 y genera el mismo lenguaje que el ejemplo de Tipo 0.
 - En general, un mismo lenguaje puede describirse mediante muchas gramáticas diferentes.
 - Una gramática determinada describe siempre un único lenguaje

Ejemplo de gramática tipo 1

- $G = (\Sigma_T = \{a, b\}, \Sigma_N = \{S\}, S, P)$
- $P = \{S ::= aSb \mid ab\}$
- $L(G) = \{a^n b^n \mid n \geq 1\}$

Ejemplo 1 Ejercicio gramática de estructura de frase

- $G' = (\Sigma_T = \{a, b\}, \Sigma_N = \{A, B, C, X, Y\}, A, P')$
- $P' = \{$
 - $A ::= aABC \mid abC$
 - $CB ::= XB$
 - $XB ::= XY$
 - $XY ::= BY$
 - $BY ::= BC$
 - $bB ::= bb$
 - $bC ::= b\}$
- El lenguaje representado es $\{a^n b^n, n \geq 1\}$

■ $G = (\Sigma_T = \{0, 1\}, \Sigma_N = \{S, A, B\}, S, P)$

• $P = \{S ::= 0B \mid 1A$

$A ::= 0 \mid 0S \mid 1AA$

$B ::= 1 \mid 1S \mid 0BB\}$

■ $L(G) = \{\text{palabras con el mismo número de ceros que unos}\}$

- $G = (\Sigma_T = \{a, b, c\}, \Sigma_N = \{S\}, S, P)$

- $P = \{S ::= aSa \mid bSb \mid c\}$

- $L(G) = \{xcx^{-1} \mid x \in \{a,b\}^*\}$

- $G = (\Sigma_T = \{a, b\}, \Sigma_N = \{S\}, S, P)$
 - $P = \{S ::= aSa \mid bSb \mid \lambda\}$

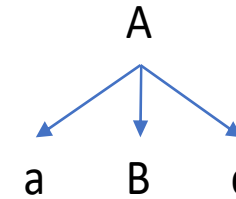
● ▪ $L(G) = \{xx^{-1} \mid x \in \{a,b\}^*\}$

Árbol de derivación

- Dada una gramática $G = (\Sigma_T, \Sigma_N, S, P)$ una derivación de una palabra “u” se puede representar mediante un árbol:

- La raíz es S

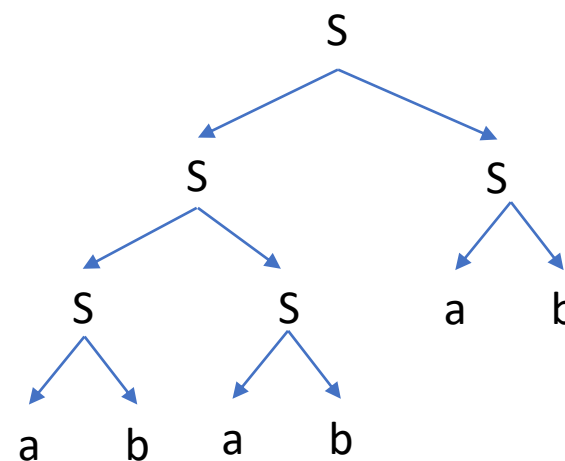
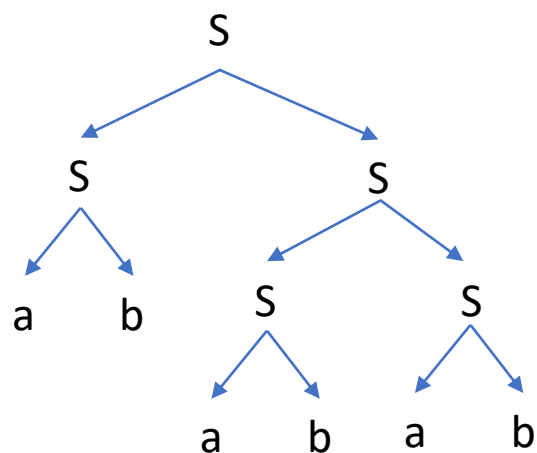
- Si se aplica a una producción del tipo $A ::= aBc$, se expresa:



- Hay tantos hijos del nodo A como símbolos tenga la parte derecha de la producción, ordenados.
- En cada paso de la derivación las hojas leídas de izquierda a derecha constituyen una **forma sentencial**.
- Si todos los vértices son símbolos de Σ_T , entonces se tiene una **sentencia** $u \in L(G)$.

Ejemplo árbol de derivación

- $G = (\Sigma_T = \{a, b\}, \Sigma_N = \{S\}, S, P = \{S ::= SS \mid ab\})$
- Sea la palabra $x = ababab \in L(G)$
- Hay dos árboles de derivación para x :



- Estos árboles corresponden a las derivaciones de la palabra $ababab$ siguientes
 - $S \rightarrow SS \rightarrow_+ \mathbf{abSS} \rightarrow_+ ababab$
 - $S \rightarrow SS \rightarrow_+ \mathbf{SSab} \rightarrow_+ ababab$

- Def **Gramática ambigua**: si tiene al menos una palabra ambigua x , es decir, con dos árboles diferentes para x , o dos derivaciones por la izquierda distintas.
- Def **Derivación por la izquierda**: Es derivación por la izquierda si y solo si en cada paso de la derivación se reemplaza el símbolo no terminal situado más a la izquierda posible.
- En el caso anterior:
$$G = (\Sigma_T = \{a, b\}, \Sigma_N = \{S\}, S, P = \{S ::= SS \mid ab\})$$
 - $S \rightarrow SS \rightarrow abS \rightarrow abSS \rightarrow ababS \rightarrow ababab$
 - $S \rightarrow SS \rightarrow SSS \rightarrow abSS \rightarrow ababS \rightarrow ababab$
 - La gramática es ambigua porque $x = ababab$ es palabra ambigua, tiene dos derivaciones por la izquierda diferentes o dos árboles de derivación distintos.

2 derivaciones por la izquierda distintas

Gramáticas bien formadas. Depuración de gramáticas

- La depuración de una gramática $G = (\Sigma_T, \Sigma_N, S, P)$ consiste en una serie de transformaciones tras las cuales se obtiene otra gramática $G' = (\Sigma_T, \Sigma'_N, S', P')$ equivalente, $L(G) = L(G')$, y que no contiene:
 1. Axioma inducido (en el caso de que $S ::= \lambda \in P$)
 2. Reglas innecesarias
 3. Reglas no generativas
 4. Símbolos inaccesibles
 5. Reglas de red denominación
 6. Reglas reductoras
- Def **gramática bien formada o depurada**: Una gramática se dice que es una gramática bien formada si no posee reglas innecesarias, reglas no generativas, símbolos no accesibles, reglas de red denominación ni reglas reductoras. Si, además, posee la regla $S ::= \lambda$ tampoco se permite la existencia del axioma inducido.

Depuración de gramáticas. Reglas innecesarias

- Son de la forma $A ::= A$ con $A \in \Sigma_N$
- Estas reglas se eliminan.

Depuración de gramáticas. Símbolos inaccesibles

- Hay que eliminarlos para obtener una gramática conexa, es decir que todos los símbolos **no terminales** sean **accesibles desde el axioma**.
 - $\nexists A \in \Sigma_N / S \rightarrow^* xAy \ \& \ x, y \in \Sigma^*$
- Para su eliminación se utiliza una matriz booleana y el siguiente **algoritmo**:
 1. Se marca el axioma
 2. Se marcan los símbolos no terminales inducidos por el axioma o por variables ya marcadas
 3. Si no se pueden marcar más símbolos, los no marcados son inaccesible. Se eliminan esos símbolos y las producciones dónde están.

Ejemplo eliminación símbolos inaccesibles

■ $P = \{$

$S ::= aB \mid bC$

$B ::= bA \mid b$

$C ::= cC \mid c$

$D ::= dC \}$

■ Marco símbolos accesibles desde S. En cola: B, C

	S	A	B	C	D
S	x		x	x	
A					
B					
C					
D					

Ejemplo eliminación símbolos inaccesibles

- Marco símbolos accesibles desde B. En cola: C, A

	S	A	B	C	D
S	x		x	x	
A					
B		x			
C					
D					

$P = \{$
 $S ::= aB \mid bC$
 $B ::= bA \mid b$
 $C ::= cC \mid c$
 $D ::= dC \}$

- Marco símbolos accesibles desde C. En cola: A

	S	A	B	C	D
S	x		x	x	
A					
B		x			
C				x	
D					

Ejemplo eliminación símbolos inaccesibles

- Marco símbolos accesibles desde A. En cola: $\emptyset$

	S	A	B	C	D
S	x		x	x	
A					
B		x			
C				x	
D					

$P = \{$
 $S ::= aB \mid bC$
 $B ::= bA \mid b$
 $C ::= cC \mid c$
 $D ::= dC \}$

- El símbolo D no es accesible desde el axioma, se elimina.

- $P' = \{$

$S ::= aB \mid bC$

$B ::= bA \mid b$

$C ::= cC \mid c \}$

Depuración de gramáticas. Reglas superfluas o no generativas

- Las reglas superfluas son las producciones que tienen en sus cadenas símbolos no terminales sin poder ser eliminadas para obtener palabras del lenguaje, es decir, símbolos no generativos.
- Ejemplo: $B ::= bA$ en derivación directa o de mayor longitud, $L(B) = \emptyset$, A no genera ningún lenguaje.
- Para eliminarlas se utiliza el **algoritmo**:
 - Sea $P_G = \{A ::= x / \exists S \rightarrow_+ uAV \rightarrow_+ u\mathbf{x}v \rightarrow_+ y \in \Sigma_T^*\}$ el conjunto de las producciones generativas.
 - Paso 1: $P_0 = N_0 = \emptyset, i = 0$
 - Paso 2: $P_{i+1} = \{A ::= x / x \in (\Sigma_T \cup N_i)^*\}$
 $N_{i+1} = \{A \in \Sigma_N / \exists A ::= x \in P_{i+1}\}$
 - Paso 3: Si $N_{i+1} \neq N_i$ hacer $i = i+1$ y volver al paso 2
 - Paso 4: Si $N_{i+1} = N_i$ entonces $P_G = P_{i+1}$ Fin.
 - Se eliminarán todas las producciones que no se encuentre en el conjunto P_G

Ejemplo de eliminación de reglas superfluas o no generativas

■ $P = \{$

$S ::= AB \mid A \mid CS1 \mid 0E$

$A ::= 0AS \mid \lambda \mid A0 \mid C$

$B ::= B1 \mid 1$

$E ::= E1 \}$

$\Sigma_T = \{0, 1\}$

Paso 2: $P_{i+1} = \{A ::= x \mid x \in (\Sigma_T \cup N_i)^*\}$

$N_{i+1} = \{A \in \Sigma_N \mid \exists A ::= x \in P_{i+1}\}$

■ Empieza: $i=0$, $P_0 = N_0 = \emptyset$

■ Iteración 1

• $P_1 = \{B ::= 1, A ::= \lambda\}$

• $N_1 = \{A, B\}$ (es distinto de N_0 así que seguimos)

Producciones que a la derecha tienen solo
terminales o símbolos en N_i siendo $N_0 = \emptyset$

Ejemplo de eliminación de reglas superfluas o no generativas

- $P = \{$
 $S ::= AB \mid A \mid CS1 \mid 0E$
 $A ::= 0AS \mid \lambda \mid A0 \mid C$
 $B ::= B1 \mid 1$
 $E ::= E1 \}$

Paso 2: $P_{i+1} = \{A ::= x \mid x \in (\Sigma_T \cup N_i)^*\}$
 $N_{i+1} = \{A \in \Sigma_N \mid \exists A ::= x \in P_{i+1}\}$

$N_1 = \{A, B\}$

■ Iteración 2

- $P_2 = \{B ::= 1, A ::= \lambda,$
 $B ::= B1, A ::= A0, S ::= AB, S ::= A\}$
- $N_2 = \{A, B, S\}$ (es distinto de N_1 así que seguimos)

Producciones que a la derecha tienen solo
terminales o símbolos en N_i siendo $N_1 = \{A, B\}$

Ejemplo de eliminación de reglas superfluas o no generativas

- $P = \{$
 $S ::= AB \mid A \mid CS1 \mid 0E$
 $A ::= 0AS \mid \lambda \mid A0 \mid C$
 $B ::= B1 \mid 1$
 $E ::= E1 \}$

Paso 2: $P_{i+1} = \{A ::= x \mid x \in (\Sigma_T \cup N_i)^*\}$
 $N_{i+1} = \{A \in \Sigma_N \mid \exists A ::= x \in P_{i+1}\}$

$N_1 = \{A, B\}$

$N_2 = \{A, B, S\}$

■ Iteración 3

- $P_3 = \{B ::= 1, A ::= \lambda,$
 $B ::= B1, A ::= A0, S ::= AB, S ::= A$
 $A ::= 0AS\}$

- $N_3 = \{A, B, S\} = N_2$ se para el algoritmo

- Se eliminan las producciones que no estén en $P_G (= P_3)$

Producciones que a la derecha tienen solo terminales o símbolos en N_i siendo $N_2 = \{A, B, S\}$

- $P' = \{$
 $S ::= AB \mid A$
 $A ::= 0AS \mid \lambda \mid A0$
 $B ::= B1 \mid 1 \}$

Depuración de gramáticas. Regla de redenominación

- Eliminar las reglas de redenominación: $A ::= B$ / $A, B \in \Sigma_N$
- El método es del de simplificación de derivaciones:
 - Si P_1 contiene
 - $\{ A ::= B \mid \dots$
 $B ::= x \mid \dots \}$
 - Entonces P_2 contiene (Por cada regla de tipo $B ::= x$)
 - $\{ A ::= x \mid \dots$
 $B ::= x \mid \dots$
 $\dots \} \rightarrow$ Se ha eliminado $A ::= B$
 - Si P_1 contiene
 - $\{ A ::= B_1 \mid \dots$
 $B_1 ::= B_2 \mid \dots$
 $B_2 ::= \dots$
 $\dots ::= B_n \mid \dots$
 $B_n ::= x \mid \dots \}$
 - Entonces P_2 contiene
 - $\{ A ::= x \mid \dots$
 $B ::= x \mid \dots$
 $\dots \} \rightarrow$ Se han eliminado de $A ::= B_1$ a $B_{n-1} ::= B_n$

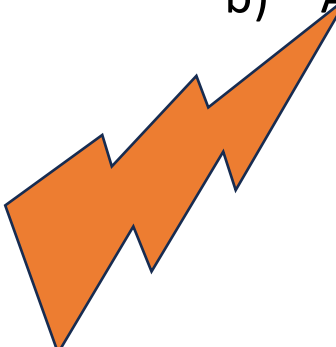
Depuración de gramáticas. Reglas reductoras

- Hay que quitar las reglas de la forma $A ::= \lambda$ con $A \neq S$
 - Si $\lambda \in L(G)$ entonces $S ::= \lambda$
 - No se admite una producción que tenga λ desde cualquier otra variable.

- Forma de eliminarlas:

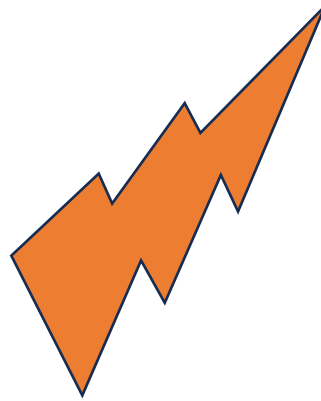
- Siempre que x e $y \neq \lambda$
- Si x e $y = \lambda$
 - a) Hacer red denominación antes
 - o
 - b) Añadir $B ::= \lambda$ (cuidado si es la parte izquierda es el axioma)

Si se tiene: $A ::= \lambda$ $B ::= xAy$	$\in P_1$	$\Rightarrow$	$B ::= xAy$ $B ::= xy$	$\in P_2$
Si se tiene: $A ::= \lambda$ $B ::= xAyAz$	$\in P_1$	$\Rightarrow$	$B ::= xAyAz$ $B ::= xyAz$ $B ::= xAyz$ $B ::= xyz$	$\in P_2$



Depuración de gramáticas. Axioma inducido

- Si en la gramática está la producción $S ::= \lambda$, y S aparece en la **derecha** de una producción, entonces se dice que el **axioma** está **inducido**.
- Si $G = (\Sigma_T, \Sigma_N, S, P)$ con axioma inducido, se construye otra gramática
 - $G' = (\Sigma_T, \Sigma'_N = \{\Sigma_N \cup S'\}, S', P')$ con $P' = P \cup \{S' ::= S\}$. En esta nueva gramática el axioma no está inducido.
- Después habrá que aplicar reglas reductoras y red denominación.



- No hay orden establecido.
- Aconsejable:
 - realizar primero los pasos que implican eliminación de reglas y símbolos (**reglas innecesarias, reglas no generativas y símbolos no accesibles**)
 - posteriormente eliminar el **axioma inducido** (en el caso de que $S ::= \lambda \in P$ y S aparezca a la derecha de alguna regla) y las **reglas reductoras** y de **redenominación**
 - Sin embargo, quizás haya que repetir algún proceso varias veces hasta conseguir la depuración de una gramática.

- Dada la $G = (\Sigma_T = \{a, b\}, \Sigma_N = \{S, A, B, C, E\}, S, P)$
- $P = \{$
 - $S ::= ABA \mid ABE$
 - $A ::= a \mid \lambda \mid E$
 - $B ::= A \mid b$
 - $E ::= Ea$
 - $C ::= a\}$
- Depurar la gramática

Para casa

- Dada la $G = (\Sigma_T = \{a, b\}, \Sigma_N = \{S, A, B, C, E\}, S, P)$

- $P = \{$

$S ::= ABA \mid ABE$

$A ::= a \mid \lambda \mid E$

$B ::= A \mid b$

$E ::= Ea$

$C ::= a\}$

- Eliminar símbolos no accesibles:

- $P = \{$

$S ::= ABA \mid ABE$

$A ::= a \mid \lambda \mid E$

$B ::= A \mid b$

$E ::= Ea$

~~$C ::= a\}$~~

Ejercicio depurar gramáticas 1

- $P = \{$
 - $S ::= ABA \mid ABE$
 - $A ::= a \mid \lambda \mid E$
 - $B ::= A \mid b$
 - $E ::= Ea\}$
- Eliminar reglas no generativas:
 - Empieza: $i=0$, $P_0 = N_0 = \emptyset$
 - Iteración 1
 - $P_1 = \{A ::= a, A ::= \lambda, B ::= b\}$
 - $N_1 = \{A, B\}$ (es distinto de N_0 así que seguimos)
 - Iteración 2
 - $P_2 = \{A ::= a, A ::= \lambda, B ::= b, S ::= ABA, B ::= A\}$
 - $N_2 = \{A, B, S\}$ (es distinto de N_1 así que seguimos)
 - Iteración 3
 - $P_3 = \{A ::= a, A ::= \lambda, B ::= b, S ::= ABA, B ::= A\}$
 - $N_3 = \{A, B, S\}$ (es igual de N_2 así que paramos)
 - $P'' = \{$
 - $S ::= ABA$
 - $A ::= a \mid \lambda$
 - $B ::= A \mid b\}$

Ejercicio depurar gramáticas 1

- $P = \{S ::= ABA$

$A ::= a \mid \lambda$

$B ::= A \mid b\}$

- Eliminar renombrado $B ::= A$

- $P = \{S ::= ABA$

$A ::= a \mid \lambda$

$B ::= a \mid \lambda \mid b\}$

- Eliminar reductora $A ::= \lambda$

- $P = \{S ::= ABA \mid \mathbf{BA} \mid \mathbf{AB} \mid \mathbf{B}$

$A ::= a$

$B ::= a \mid \lambda \mid b\}$

- Eliminar renombrado $S ::= B$

- $P = \{S ::= ABA \mid BA \mid AB \mid a \mid \lambda \mid b$

$A ::= a$

$B ::= a \mid \lambda \mid b\}$

- Eliminar reductora $B ::= \lambda$

- $P = \{S ::= ABA \mid BA \mid AB \mid a \mid \lambda \mid b \mid \mathbf{AA} \mid \mathbf{A}$

$A ::= a$

$B ::= a \mid b\}$

- Eliminar renombrado $S ::= A$

- $P = \{S ::= ABA \mid BA \mid AB \mid a \mid \lambda \mid b \mid AA$

$A ::= a$

$B ::= a \mid b\}$

- Dada la $G = (\Sigma_T = \{0, 1\}, \Sigma_N = \{S, A, B, C, D\}, S, P)$
- $P = \{$
 - $S ::= 0AA \mid 1AB \mid 0AC \mid 1BD \mid A$
 - $A ::= 00 \mid 01 \mid 10 \mid 11 \mid A \mid \lambda$
 - $B ::= 1B \mid 1C \mid 1D \mid 1 \mid 1S$
 - $C ::= 0C \mid 1D \mid B\}$
- Depurar la gramática

Para casa

Gramáticas tipo 3. Gramáticas lineales

- Lineales por la **izquierda**
 - Reglas de producción del tipo
 - $A ::= a; A ::= Va$ dónde
 - $A, V \in \Sigma_N$ (A y V son no terminales)
 - $a \in \Sigma_T$ (a es terminal)
 - No son compresoras
 - Para que $\lambda \in L(G)$ se debe añadir $S ::= \lambda$
 - Los lenguajes representados por estas gramáticas se llaman “lenguajes regulares” (L_3)
 - Los lenguajes regulares se representan por medio de expresiones regulares.
- Lineales por la **derecha**
 - Reglas de producción del tipo
 - $A ::= a; A ::= aV$ dónde
 - $A, V \in \Sigma_N$ (A y V son no terminales)
 - $a \in \Sigma_T$ (a es terminal)

■ $G = (\Sigma_T = \{0, 1\}, \Sigma_N = \{S, A\}, S, P)$

- $P = \{S ::= 0S \mid 0A$

- $A ::= 1A \mid 1\}$

● ■ Gramática lineal derecha.

- $L(G) = \{0^m 1^n \mid m, n \geq 1\}$

- $L(G) = 00^*11^* = 0^*01^*1$ (expresión regular)

- $G = (\Sigma_T = \{0, 1\}, \Sigma_N = \{A, B\}, A, P)$
 - $P = \{A ::= B1 \mid 1$
 $B ::= A0\}$

- $G = (\Sigma_T = \{0, 1\}, \Sigma_N = \{A, B\}, A, P)$
 - $P = \{A ::= 1B \mid 1$
 $B ::= 0A\}$

- ■ Gramática lineal izquierda.
 - $L(G) = \{1(01)^n \mid n \geq 0\}$
 - $L(G) = (10)^*1 = 1(01)^*$

- ■ Gramática lineal derecha.
 - $L(G) = \{1(01)^n \mid n \geq 0\}$

- $G = (\Sigma_T = \{0, 1\}, \Sigma_N = \{A, B\}, A, P)$

- $P = \{A ::= 1A \mid 0B$

- $B ::= 0B \mid 0\}$

- Gramática lineal derecha.

- $L(G) = 1^*000^*$

Definición equivalencia entre gramáticas GLI y GLD

- (recordatorio) Se dice que dos gramáticas son equivalentes si describen el mismo lenguaje
- Dadas las gramáticas G y G'
 - $G \equiv G' \Leftrightarrow L(G) = L(G')$
- **Teorema:** Para toda gramática lineal derecha existe otra gramática lineal izquierda equivalente.
Y para toda gramática lineal izquierda existe otra gramática lineal derecha equivalente.

- Construir las gramáticas que generan los siguientes lenguajes, indicando de qué tipo es la gramática obtenida y cuando sea posible expresar L mediante expresión regular.
 1. $L = \{a^m b^n / m > 0, n \geq 0\}$
 2. $L = \{a^m b^n / n > m\}$
 3. $L = \{a^m b^n c^k / m = n + k\}$
 4. L, lenguaje formado por las palabras sobre el alfabeto $\Sigma = \{a, b\}^*$ tales que inmediatamente después de una “a” hay una “b”.

- Construir las gramáticas que generan los siguientes lenguajes, indicando de qué tipo es la gramática obtenida y cuando sea posible expresar L mediante expresión regular.

1. $L = \{a^m b^n \mid m > 0, n \geq 0\}$

$L = aa^*b^*$, es lenguaje regular.

La gramática que lo genera tiene que ser gramática lineal

Una gramática lineal derecha que lo genera es:

$$G = (\Sigma_T = \{a, b\}, \Sigma_N = \{S, A\}, S, P)$$

$$P = \{S ::= aS \mid aA \mid a$$

$$A ::= bA \mid b \}$$

- Construir las gramáticas que generan los siguientes lenguajes, indicando de qué tipo es la gramática obtenida y cuando sea posible expresar L mediante expresión regular.

2. $L = \{a^m b^n \mid n > m\}$

Gramática tipo 2 que lo genera.

$$G = (\Sigma_T = \{a, b\}, \Sigma_N = \{S\}, S, P)$$

$$P = \{S ::= aSb \mid Sb \mid b\}$$

- Construir las gramáticas que generan los siguientes lenguajes, indicando de qué tipo es la gramática obtenida y cuando sea posible expresar L mediante expresión regular.

3. $L = \{a^m b^n c^k \mid m = n + k\}$

Gramática tipo 2 que lo genera.

$$G = (\Sigma_T = \{a, b, c\}, \Sigma_N = \{S, A\}, S, P)$$

$$P = \{S ::= aSc \mid A$$

$$A ::= aAb \mid \lambda \}$$

- Construir las gramáticas que generan los siguientes lenguajes, indicando de qué tipo es la gramática obtenida y cuando sea posible expresar L mediante expresión regular.

4. **L, lenguaje formado por las palabras sobre el alfabeto $\Sigma = \{a, b\}^*$ tales que inmediatamente después de una “a” hay una “b”.**

Gramática **tipo 2** que lo genera.

$$G = (\Sigma_T = \{a, b\}, \Sigma_N = \{S\}, S, P)$$

$$P = \{S ::= bS \mid abS \mid \lambda\}$$

¿Expresión regular?

Gramática **tipo 3** que lo genera.

$$G = (\Sigma_T = \{a, b\}, \Sigma_N = \{S, A\}, S, P)$$

$$P = \{S ::= bS \mid aA \mid \lambda$$

$$A ::= bS\}$$

Ejercicios diseño de gramática → Compiladores 1

- Nombres de Variables en un lenguaje de programación. **Diseña la G3 que genera:**
 - Los nombres de las **Variables** están formados por **letras minúsculas (a..z)** y **dígitos (0..9)**, **empezando siempre** por una **letra**.
 - Ejemplos.
 - Válidos: **aux51**, **mi3variable**, **cont**
 - Incorrectos: **5aux**, **aux_51**

$G = (\Sigma_T = \{a, b, c..z, 0, 1, 2, ..9\}, \Sigma_N = \{S, A\}, S, P)$

$P = \{S ::= a \mid b \mid c... \mid z \mid aA \mid bA \dots \mid zA$

$A ::= a \mid b \mid c... \mid z \mid 0 \mid 1... \mid 9 \mid aA \mid bA \dots \mid zA \mid 0A \mid 1A \dots \mid 9A \}$

Ejercicios diseño de gramática → Compiladores 2

- Nombres de Variables en un lenguaje de programación
- Los nombres de las **Variables** están formados por **letras minúsculas (a..z)**, **dígitos (0..9)** y **subrayados (_)**, **empezando siempre** por una **letra**, y **no** pueden **acabar** en **subrayado**.
- Ejemplos.
 - Válidos: `aux_51`, `mi___variable`.
 - Incorrectos: `5aux`, `mi__variable_`

Para casa

Ejercicios diseño de gramática → Extra 1 – Skin Care

- Rutina de *skin care* (al menos 1 día completo)
 - Cada día la rutina tiene un componente de mañana y uno de noche
 - Por la mañana siempre hay que lavarse la cara antes de elegir una opción: vitamina B3 (niacinamide) o vitamina C o alfa-hidroxiácidos (AHA).
 - Opcionalmente, después de la elección se puede usar crema hidratante.
 - Al final de la rutina hay que usar crema solar, al menos una vez y repetir tantas veces como sea necesario al cabo del día.
 - Si en la noche anterior se han usado ácidos retinoicos, por la mañana se puede usar exfoliante físico, que además sustituye a lavarse la cara.
 - Por la noche siempre hay que lavarse la cara antes de elegir una opción: ácido retinoico o alfa-hidroxiácidos.
 - Después de la elección hay que usar crema hidratante.
 - Entre productos hay que dejar pasar 1 minuto.
- Descargo de responsabilidad: este es un ejercicio ficticio, no una recomendación médica.

- <https://short.upm.es/ad71c>

